Lab 1 Linear Regression

MACHINE LEARNING FOR THE BUILT ENVIRONMENT

GEO5017

Authors:

Xinya Bi (6195350)
Hongyu Ye (6286240)
Xu Wang (6235379)

March 5, 2025

Contents

1 Question 1

(1) We need to plot the data points to explore the possible relationships between independent and dependent variables.

(2) Parameters can minimize the sum of squared errors between predicted values and observed values.

2 Question 2

2.1 Trajectory Plotting

We use matplotlib to plot the trajectory of the drone, see Figure 1.

2.2 Polynomial Regression

To solve the time and position problem, we first design a gradient descent algorithm that can handle simple and polynomial linear regression models (Keep the first 3 digits after the decimal point).

1. Starting with an initial parameter guess.
2. In each iteration
   (a) calculating the step size by multiplying learning rate and gradient. Gradient is the derivative of the loss function with respect to every unknown parameters.
   (b) Checking if all elements of the step size are smaller than the threshold. If so, the algorithm has converged and exits early.
   (c) Updating the new parameters by subtracting the step size from the current parameters.

2.2.1 Constant Speed Model

Best Parameters (Shown in Figure 2): $Learning\ Rate = 0.01$, $Max\ Iterations = 200$, $threshold = 0.001$

$$x_{pos} = 1.424 - 0.431 \cdot t \quad \text{SSE: } 8.330$$
$$y_{pos} = 2.081 - 0.580 \cdot t \quad \text{SSE: } 6.024$$
$$z_{pos} = 0.754 + 0.493 \cdot t \quad \text{SSE: } 1.634$$

After testing different parameter values (learning rates, iteration counts, residuals), we found minimal differences in results. We decided the best parameters for the model are $Learning\ Rate = 0.01$, $Max\ Iterations = 200$, $threshold = 0.001$. It has a few advantages:(1) It has the lowest residual errors across 3 dimensions. (2) It converges after about 150 iterations. The speed is $x_v = -0.431$,$y_v = -0.580$, $z_v = 0.493$

2.2.2 Constant Acceleration Model

(1) Methods

(A) Establishing the Model
Assuming constant acceleration, the motion of a drone in 3-dimensional space can be described by the following second-order polynomial equations in the x, y, and z directions:

$$\begin{cases} x(t) = a_x + b_x\, t + c_x\, t^2, \\ y(t) = a_y + b_y\, t + c_y\, t^2, \\ z(t) = a_z + b_z\, t + c_z\, t^2. \end{cases}$$

Here, (a_x, b_x, c_x) correspond to the coefficients in the x direction. The same applies to (a_y, b_y, c_y) and (a_z, b_z, c_z).

(2) Results

We set the maximum iterations to 6×10^4 and consider convergence achieved when the step size falls below 10^{-6}. The coefficients start from zero. Under these settings, we tested different learning rates α (Figure 3).

If α is too small, convergence is slow and computationally expensive; if too large, it may fail to converge. To optimize efficiency, we analyze the cost function and select the largest α before divergence (Table 1).

Our algorithm achieved a minimum SSE of 0.655 on x, 1.057 on y,and 0.760 on z. significantly outperforming the Constant Speed Model due to the superior fitting capability of a quadratic polynomial.

(3) Improvement

A major drawback of the algorithm is selecting an appropriate learning rate is needed, as small variations in α can drastically impact results, making manual tuning inefficient.

To address this, we implemented a decaying learning rate, starting with a relatively larger α that gradually decreases during gradient descent. Our approach halves α at intervals, eliminating manual tuning while maintaining effective convergence (Figure 4).

In addition, by observing the velocity dependence of the metadata, and the heading angle in the x-y plane and its changes (Figure 5), we consider taking into account the angular velocity and the covariance constraints of x-y-z when fitting the model.Considering the complexity of calculation and operation, we use pytorch to optimize the process.

2.2.3 Location Prediction

Our improved function yields the following fitted equations and corresponding fitting performance (Figure 6):

$$x(t) = 5.516 - 3.477 \cdot t + 0.434 \cdot t^2 \qquad \text{SSE: } 0.655$$
$$y(t) = -0.894 + 1.675 \cdot t - 0.324 \cdot t^2 \qquad \text{SSE: } 1.057$$
$$z(t) = 1.311 + 0.099 \cdot t + 0.055 \cdot t^2 \qquad \text{SSE: } 0.760$$

The predicted coordinates at $t = 7$ are: x = 2.424, y = -5.027, z = 4.689 .The position prediction results of the fitted function are shown in Figure 7 and Figure 9.

In addition, the results of the training simulation using the angular velocity and xyz covariance constraints show that the total SSE is optimized by about only 10^{-5}(2.471007 to 2.470989) compared to only applying the decaying learning rate. However, after applying covariance and angular velocity regularization, the model is more constrained and the optimization process is more stable and faster, thus reducing the number of iterations(Figure 8).

3 Work Allocation

- Xinya Bi (40%): gradient descent algorithm, trajectory plotting, constant speed model
- Hongyu Ye (30%): Implementation of gradient descent algorithms with improvements, constant acceleration models and improved trajectory plotting.
- Xu Wang (30%): Implementation of gradient descent algorithms with improvements in the angular velocity and xyz covariance constraints.

4 Additional Graphs and Data

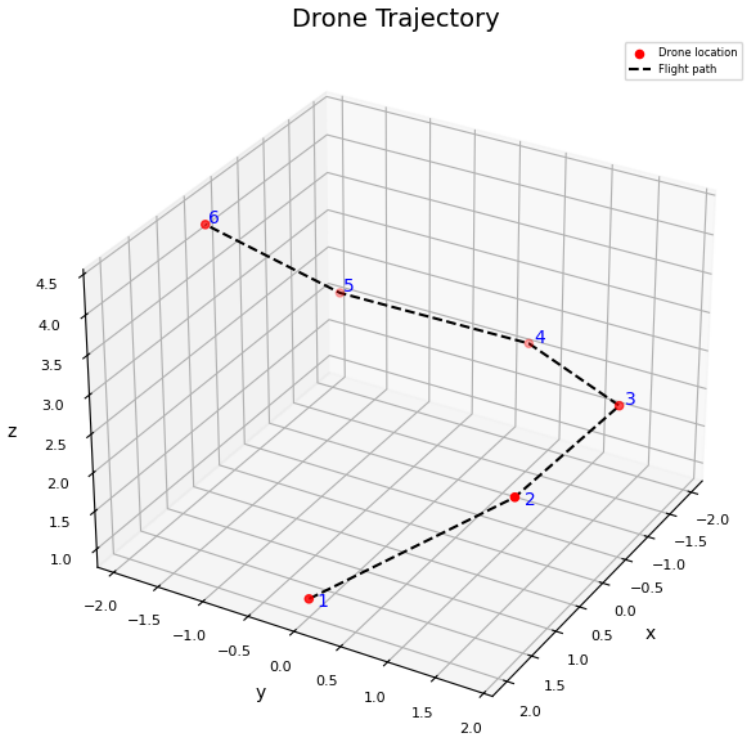

Figure 1: Drone Trajectory

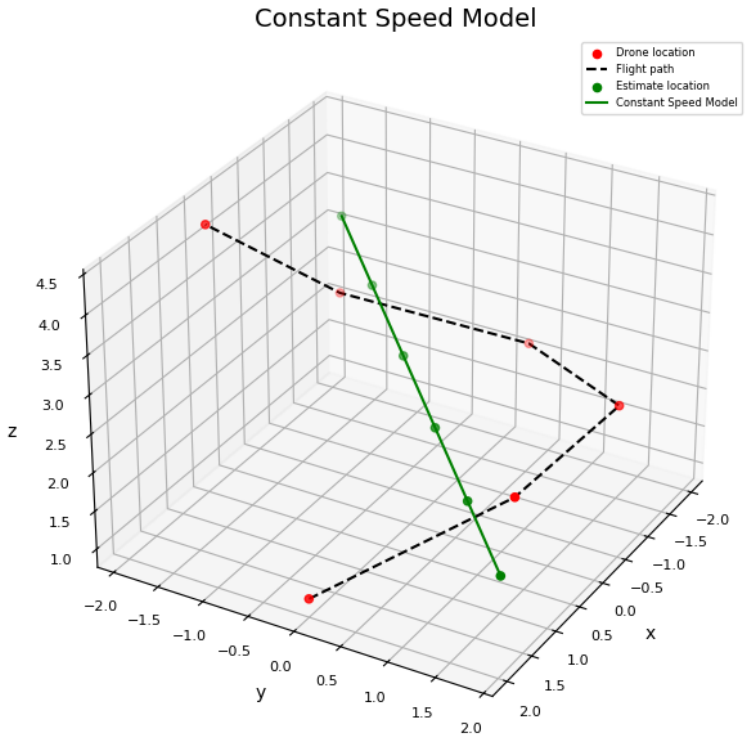

Figure 2: Constant Speed Model and the Actual Position

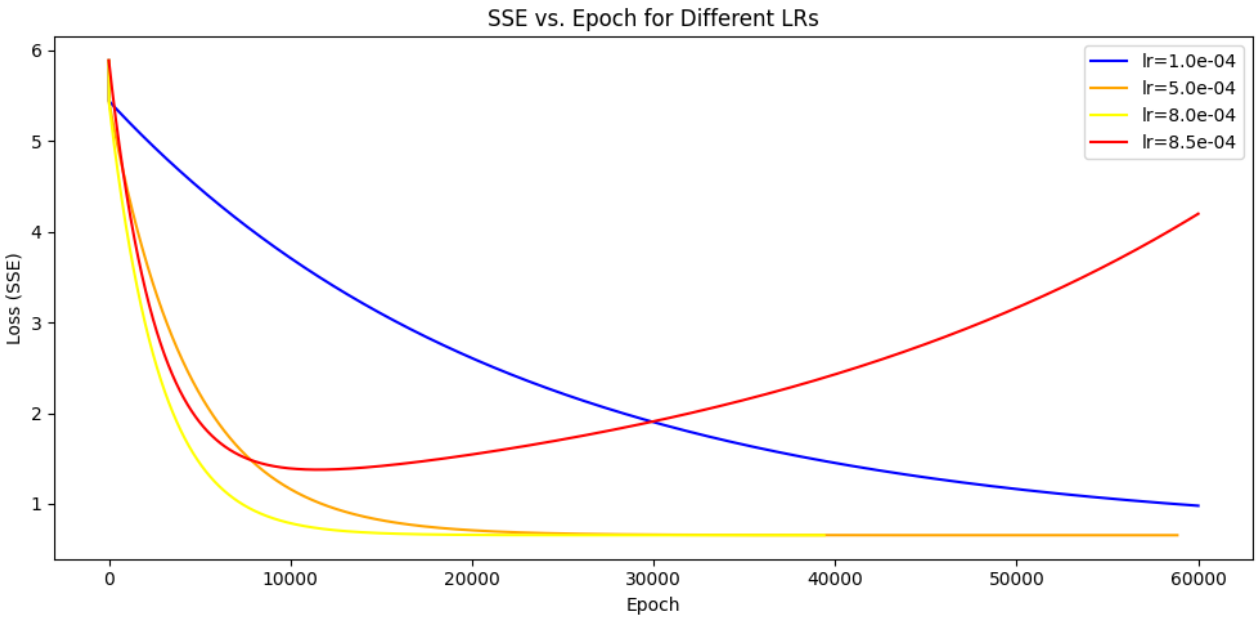

Figure 3: SSE and Epoch for Different LRs on x

Table 1: Gradient Descent Results with Different LRs on x

Learning Rate	Epochs	Step Size	Final SSE
1.00×10^{-4}	60000	3.82×10^{-5}	0.980
5.00×10^{-4}	58847	1.00×10^{-6}	0.655
8.00×10^{-4}	39399	1.00×10^{-6}	0.655
8.46×10^{-4}	60000	1.10×10^{-1}	4.199

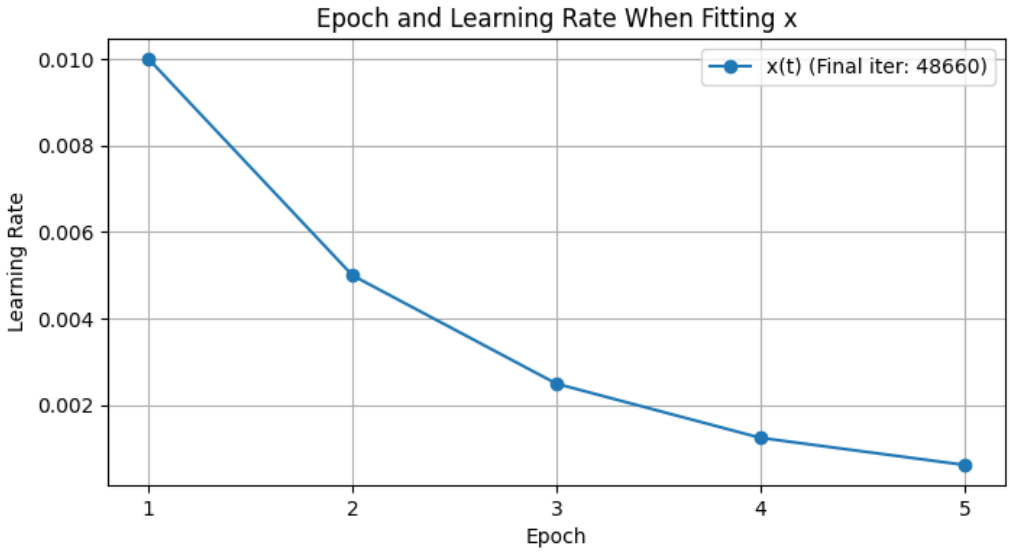

Figure 4: Decaying Learning Rate on x-position Fitting

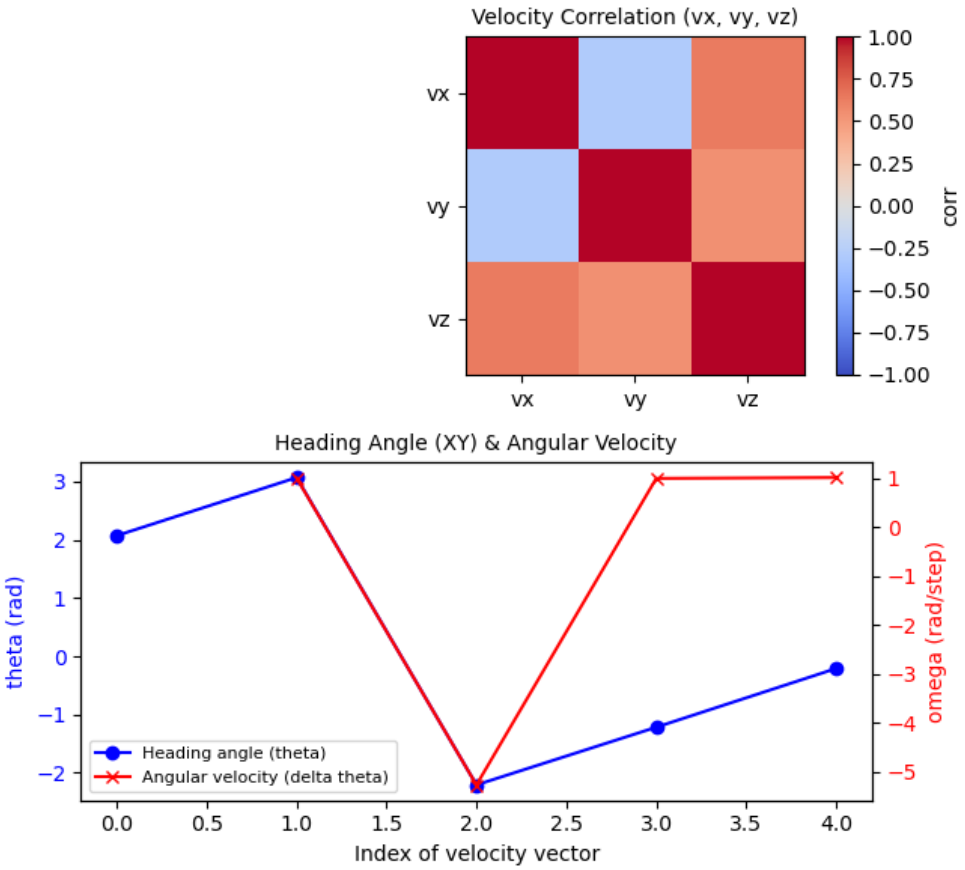

Figure 5: Velocity Correction

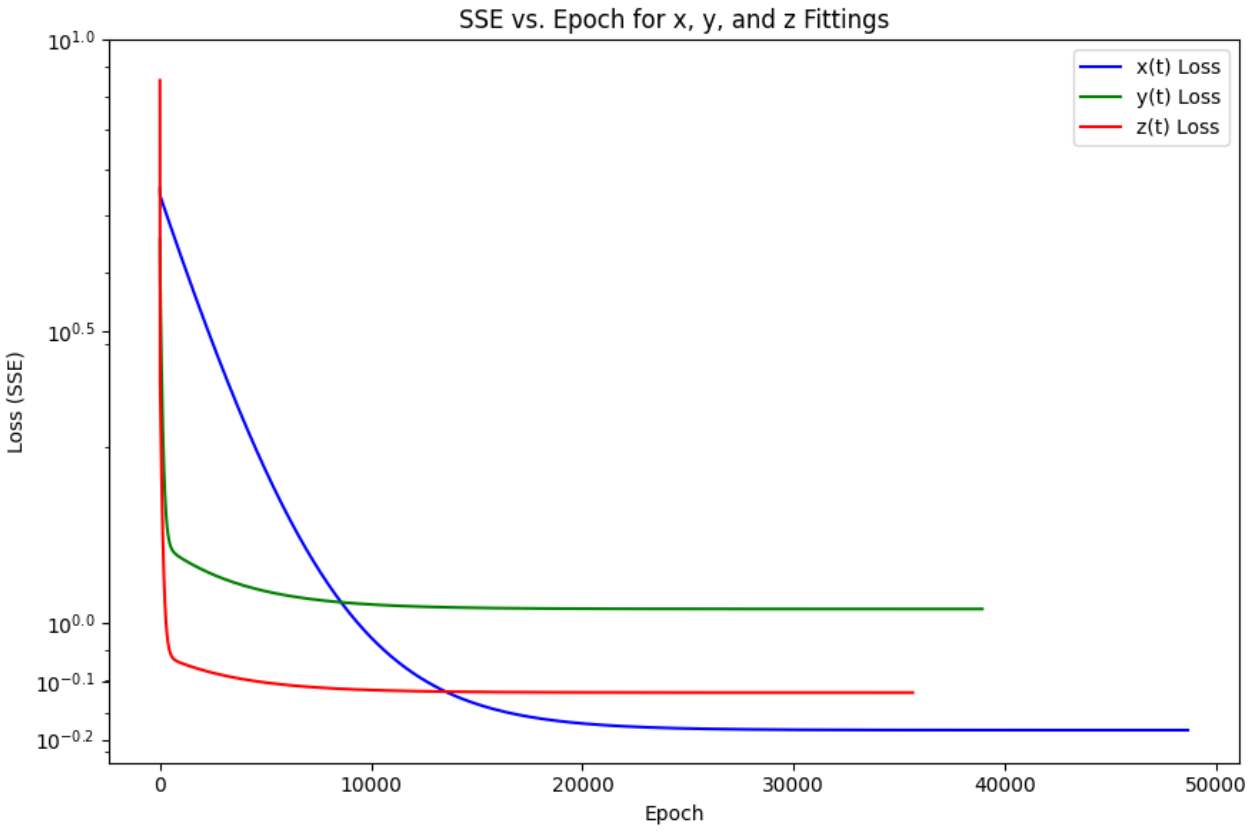

Figure 6: SSE and Epoch for x, y, and z Fittings

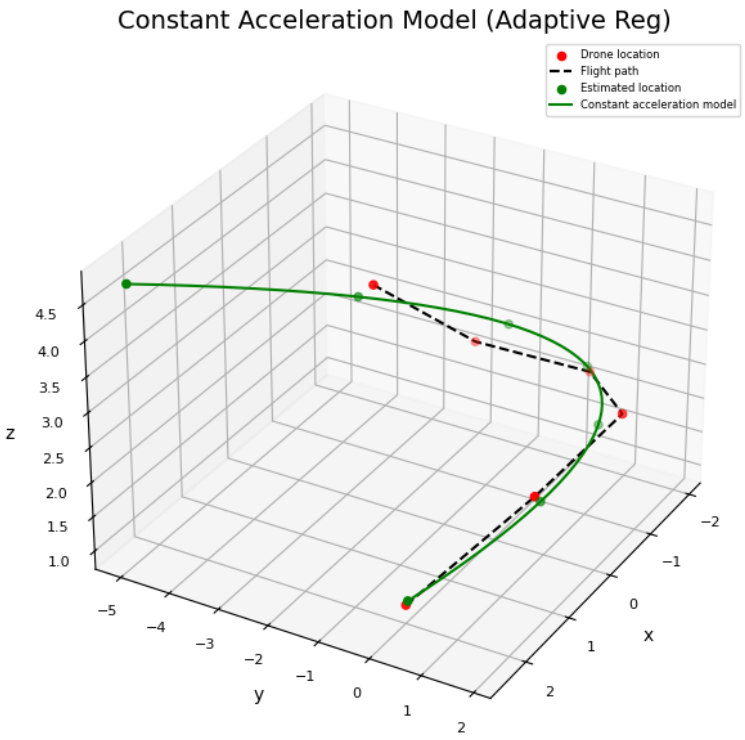

Figure 7: Constant Accelerate Model and the Actual Position (plus covariance and angular velocity regression)

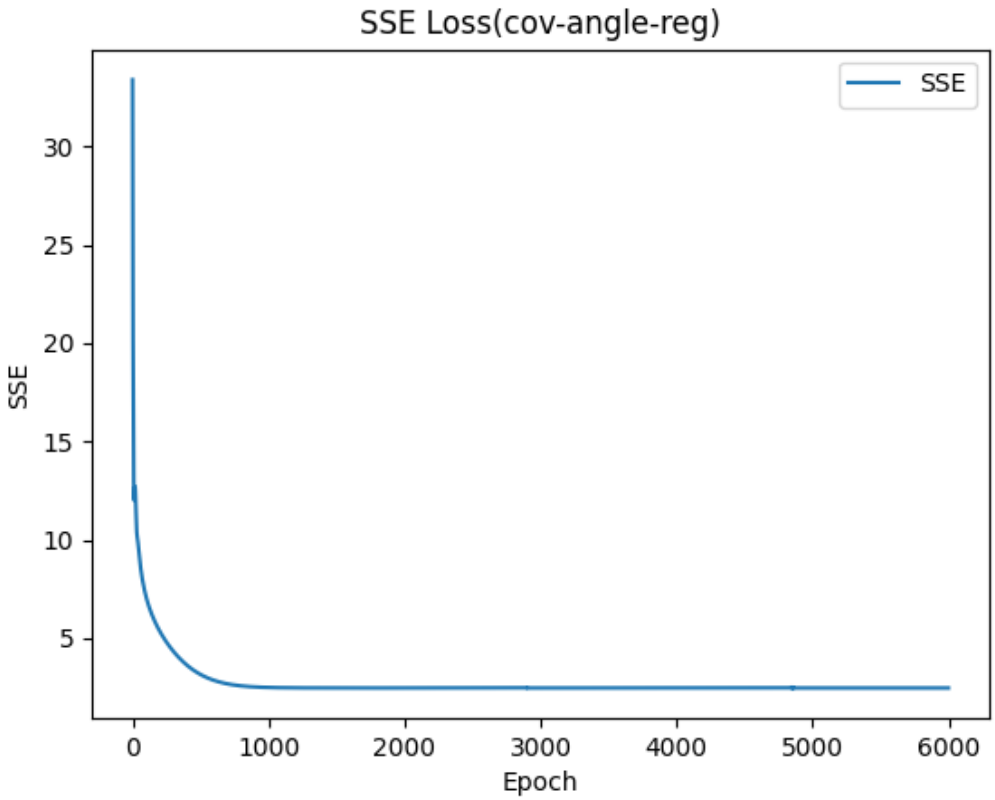

Figure 8: Constant Accelerate Model and the Actual Position(plus covariance and angular velocity regression)

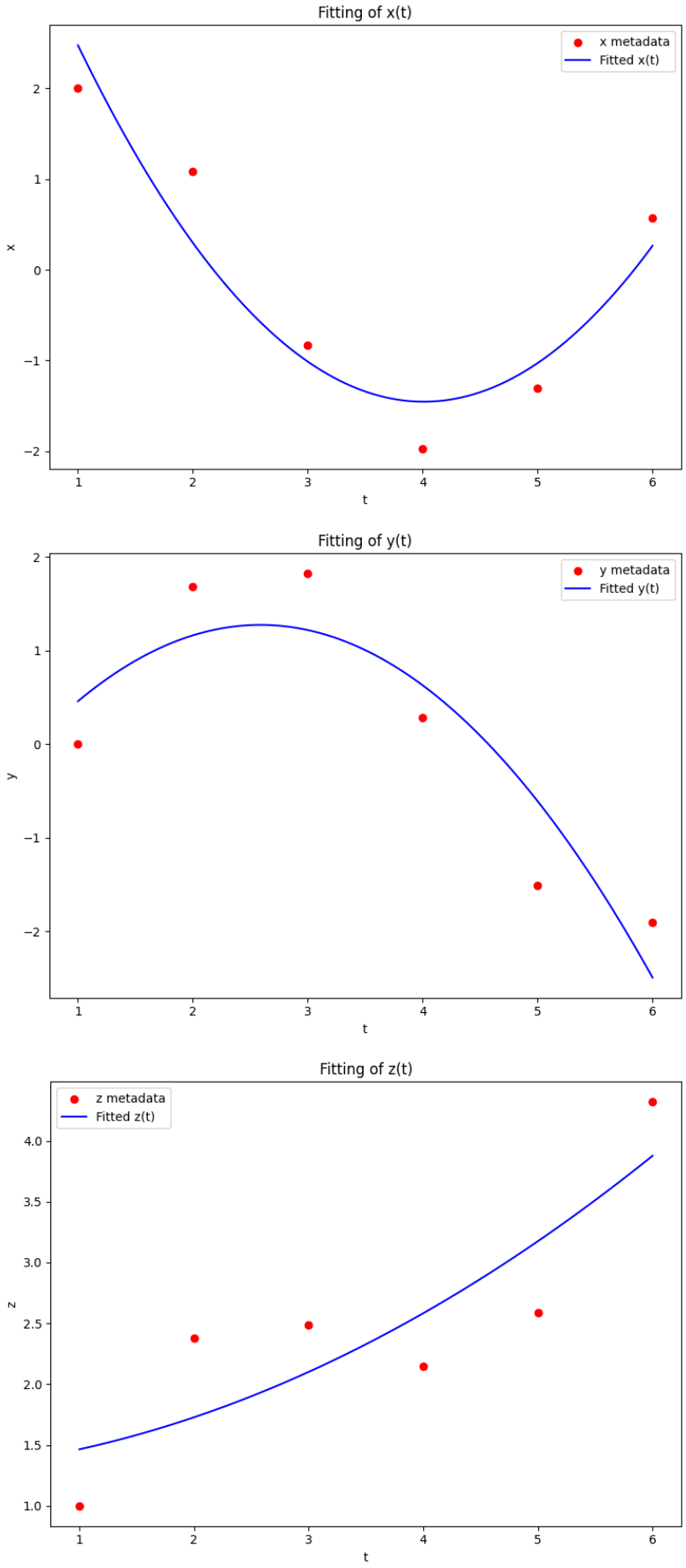

Figure 9: Fitting Results for x, y, and z